

Project Report

ATM Interface using Java

Objective

To develop a Java-based ATM Interface application using Object-Oriented Programming principles. The application enables users to perform banking operations such as balance inquiry, cash deposit, and cash withdrawal while ensuring proper input validation and encapsulation of account data. This project emphasizes modular design through separate ATM and BankAccount classes.

Software Requirements

- * IntelliJ IDEA
- * Java JDK 17+
- * Windows 10/11

Tools Used

- * Java
- * IntelliJ IDEA
- * Scanner Class

Description

The ATM Interface is a menu-driven Java console application that simulates real-world banking operations. It is designed using Object-Oriented Programming concepts, where the ATM and BankAccount are represented as separate classes. The system allows users to deposit funds, withdraw money, check account balance, and exit the application while maintaining data security through encapsulation and validating user inputs.

Features

- * Menu-driven ATM interface
- * Account balance inquiry
- * Cash deposit
- * Cash withdrawal
- * Input validation
- * Insufficient balance checking
- * Encapsulation using private variables
- * Modular class design

Java Concepts Used

- * Object-Oriented Programming
- * Classes & Objects

- * Encapsulation
- * Constructors
- * Methods
- * Scanner Class
- * Loops
- * Switch Case
- * Conditional Statements

Algorithm

1. Start the application.
2. Create a BankAccount object with an initial balance.
3. Initialize the ATM object.
4. Display the ATM menu.
5. Read the user's choice.
6. Execute the selected transaction:
 - * Check balance
 - * Deposit money
 - * Withdraw money
7. Validate the entered amount.
8. Update the account balance if the transaction is valid.
9. Display transaction status.
10. Repeat until the user selects Exit.
11. End the application.

Expected Output

- * ATM Menu
- * Account Balance
- * Deposit Confirmation
- * Withdrawal Confirmation
- * Error Messages for Invalid Transactions
- * Exit Message

Learning Outcomes

- * Understanding Object-Oriented Programming
- * Designing classes and objects
- * Implementing encapsulation
- * Developing menu-driven applications
- * Applying business logic for banking operations
- * Validating user input
- * Improving software design and debugging skills

Conclusion

The ATM Interface project successfully demonstrates the implementation of Object-Oriented Programming concepts in Java by modeling real-world banking operations.

Through separate classes for the ATM and BankAccount, the application achieves modularity, encapsulation, and maintainability while providing secure and reliable transaction handling. This project strengthened practical knowledge of Java application development and fulfilled the objectives of Project 3 in the Decode Labs Java Internship.